30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 12

Network Layer — Part 2: TLS, Pinning, OAuth2 & APIs at Scale



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 12

Why network security matters in 2026

HTTPS is the default now, so interviews moved up a level: *what do you build on top of TLS, and what can go wrong when you do?* Fintech, health and any app with accounts will probe this:

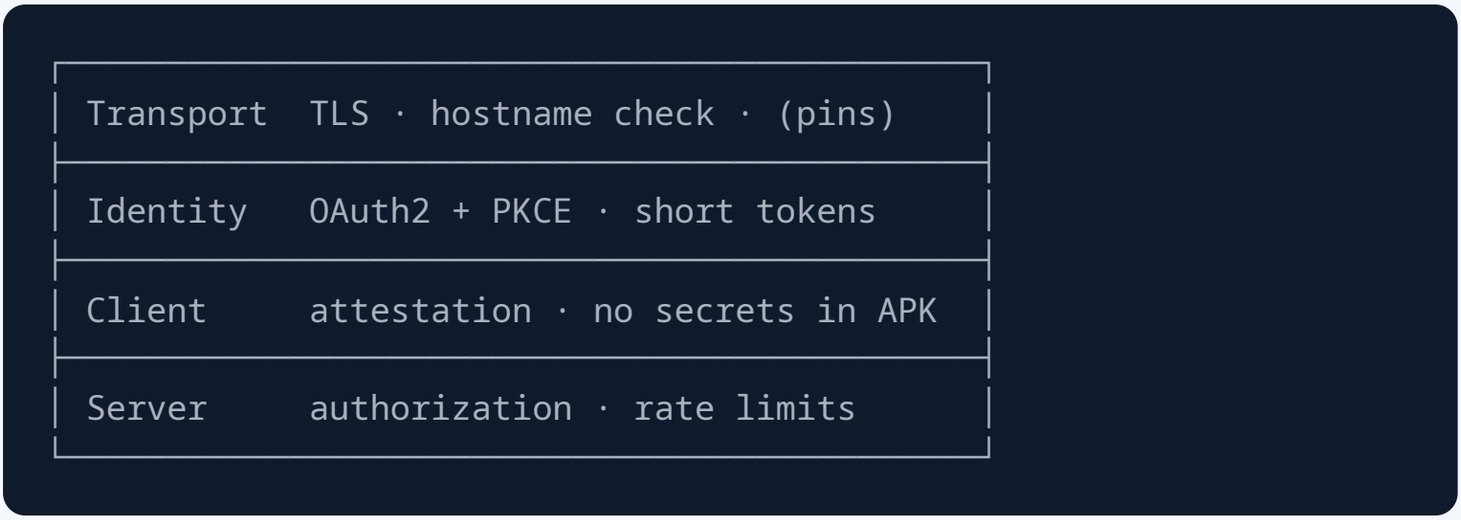
- TLS, **Network Security Config** and what they do — and don't — protect
- **Certificate pinning** as a trade-off with real outage risk, not a checkbox
- **OAuth2 + PKCE** and token refresh under concurrency — a classic senior coding question
- Secrets: why nothing in the APK is secret
- Scale: pagination, rate limits, GraphQL, gRPC, WebSockets

Level	Questions
Junior	TLS & HTTPS · Network Security Config · Auth schemes
Mid	Certificate pinning · Pinning risks · OAuth2 + PKCE · Tokens & Authenticator
Senior	Refresh under concurrency · API keys & secrets · Pagination & rate limits
Architect	Beyond REST · Real-time connections · Security architecture



Esmaeeil Moradi
@esmaeeil-moradi700

Mental model: layers of defense



- **TLS** proves you're talking to the right server — it says nothing about who the *client* is
- **Tokens** identify the user; keep them short-lived and refreshable
- **The APK is public:** the server is the only place where rules are enforced

DEFENSE IN DEPTH

Every layer assumes the one above it can fail. A good answer names the threat first, then the control.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 12



Junior

TLS & HTTPS · Network Security Config · Auth schemes



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What TLS gives you



- **Confidentiality** (encryption), **integrity** (tampering is detected), **server authentication** (certificate chain + hostname)
- Android 10+ enables TLS 1.3 by default; apps targeting API 28+ block cleartext HTTP by default

INTERVIEW TRAP

"HTTPS hides everything." — The hostname is visible (SNI, DNS). Path, query, headers and body are encrypted — but tokens in query strings still land in server logs, proxies and analytics. Send them in headers.



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. The mistakes that break TLS

INTERVIEW TRAP

Fixing *"SSL handshake failed"* with a trust-all `X509TrustManager` or a `HostnameVerifier` that returns `true`. That switches off server authentication for every call — and Google Play flags unsafe implementations. Fix the server chain (often a missing intermediate) or use Network Security Config for a dev CA.

FOLLOW-UP

"Can a user see your app's HTTPS traffic?" — On their own device, yes: with root, or a build that trusts user CAs, a proxy like mitmproxy decrypts it. TLS protects users **from others**; it doesn't hide your API from its users.

RULE OF THUMB

Treat every request as readable by the person holding the phone. Design the API so that's safe.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. Network Security Config

```
<!-- AndroidManifest.xml -->
<application
    android:networkSecurityConfig="@xml/network_security_config">

<!-- res/xml/network_security_config.xml -->
<network-security-config>
    <base-config cleartextTrafficPermitted="false" />
    <debug-overrides>
        <trust-anchors>
            <certificates src="user" />
        </trust-anchors>
    </debug-overrides>
</network-security-config>
```

- Declarative and app-wide: cleartext per domain, trust anchors, pins — no code
- Apps targeting API 24+ **don't trust user-installed CAs**; `debug-overrides` applies only to debuggable builds
- Cleartext to a local dev server? Add a `domain-config` in the **debug** source set's copy of the file



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. What interviewers check

INTERVIEW TRAP

"Trust user CAs in release so QA can use Charles." — Then any user-installed CA — a corporate proxy, a malicious profile — can read your users' traffic. That's exactly what `debug-overrides` is for.

INTERVIEW TRAP

`cleartextTrafficPermitted="true"` in `base-config` because one image CDN still serves HTTP. That opens cleartext for **every** host. Scope it with a `domain-config` for that host — better, move the CDN to HTTPS.

FOLLOW-UP

"Does OkHttp respect it?" — Yes. OkHttp uses the platform trust manager, and it refuses cleartext to hosts the policy forbids (`UnknownServiceException: CLEARTEXT communication … not permitted`).



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Authentication schemes compared

Scheme	What is sent	Identifies	Notes
Basic	`base64(user:password)` on every call	User	Encoding, not encryption; the password travels constantly
API key	A static key in a header	The app or project	Public once shipped (Q9)
Session cookie	Cookie set by the server	A session	Browser-oriented; needs CSRF protection
Bearer token	Short-lived access token	User + scopes	With a refresh token — the mobile default

INTERVIEW TRAP

"A JWT is encrypted." — A typical JWT is only **signed**: anyone can Base64-decode the payload. Never put secrets in it, and never let the app make authorization decisions from it — the server verifies the signature.

FOLLOW-UP

"Should the app read the JWT at all?" — Reading `exp` to refresh a little early is fine. Trusting its claims is not.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 12

Mid-Level

Certificate pinning · Pinning risks · OAuth2 + PKCE · Tokens & Authenticator



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Certificate pinning with OkHttp

```
val pinner = CertificatePinner.Builder()
    .add(
        "api.example.com",
        // current key
        "sha256/AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA=",
        // backup key, kept offline and not yet deployed
        "sha256/BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB=",
    )
    .build()

val client = OkHttpClient.Builder()
    .certificatePinner(pinner)
    .build()
```

- A pin is the SHA-256 of a certificate's **public key** (SPKI) — renewing a certificate with the same key keeps it valid
- The connection passes if **any** certificate in the verified chain matches a pin
- Pinning is checked **on top of** normal TLS validation, never instead of it



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Getting the pin — and what it protects

```
openssl s_client -connect api.example.com:443 \
    -servername api.example.com </dev/null \
  | openssl x509 -pubkey -noout \
  | openssl pkey -pubin -outform der \
  | openssl dgst -sha256 -binary \
  | openssl enc -base64
```

INTERVIEW TRAP

"Pinning blocks forged certificates from untrusted CAs." — Last year's framing. Normal TLS already rejects those. Pinning protects against a **trusted** CA issuing a certificate for your domain — compromised, coerced, or added to the device's system store.

INTERVIEW TRAP

"Pinning stops people from reverse-engineering our API." — On a device the attacker controls, hooking tools disable pinning in minutes. Pinning protects **users** from network attackers, not your API from its users.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Pinning risks: what to pin

Pin target	Protection	Rotation risk
Your leaf key	Strongest	High — every key change needs a new release first
Intermediate CA key	Good	Medium — breaks if the CA switches intermediates
Root CA key	Weaker	Lower — breaks if you change CA
Backup key (offline, unused)	Enables planned rotation	Always include one

- Old app versions keep their pins **forever** — a wrong rotation bricks every install that hasn't updated
- `<pin-set expiration="…">` in Network Security Config: after that date pinning is **switched off** instead of failing — a built-in safety valve
- Don't pin hosts you don't control (CDNs, analytics, third-party SDKs) — they rotate without asking you



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Should you pin at all?

INTERVIEW TRAP

"Solve rotation with dynamic pins downloaded from a pinned endpoint." — Suggested last year. The bootstrap pin is still hardcoded, and now you run a pin-distribution service that must never be wrong. It moves the risk; it doesn't remove it.

FOLLOW-UP

"Would you pin?" — Android's own documentation warns that pinning is risky for most apps. I pin only high-value targets (banking, wallets) when we own the keys: backup pin, expiration date, failure telemetry and a rotation runbook. Otherwise: solid TLS plus Certificate Transparency monitoring, which alerts you when any CA issues a certificate for your domain.

MY TAKE

Pinning is an operations commitment, not a security feature you switch on. If nobody owns the rotation calendar, don't pin.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. OAuth2 + PKCE for native apps

```
1 App      makes code_verifier (random, stays in app)
           code_challenge = BASE64URL(SHA-256(verifier))
2 Browser  /authorize?…&code_challenge=…&state=…
           user signs in on the auth server's own page
3 Redirect app receives ?code=…&state=…  (check state)
4 App      POST /token   code + code_verifier
5 Server   hashes verifier, compares → access + refresh
```

- Native apps are **public clients**: they can't keep a client secret
- PKCE binds the code to the app instance that started the flow — a stolen code is useless without the verifier
- Sign-in runs in the **system browser via Custom Tabs** (AppAuth does all of this) — never in an embedded WebView



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. PKCE in code — and the traps

```kotlin
private val flags =
    Base64.URL_SAFE or Base64.NO_PADDING or Base64.NO_WRAP

fun newCodeVerifier(): String {
    val bytes = ByteArray(32)
    SecureRandom().nextBytes(bytes)
    return Base64.encodeToString(bytes, flags)
}

fun codeChallenge(verifier: String): String {
    val digest = MessageDigest.getInstance("SHA-256")
        .digest(verifier.toByteArray(Charsets.US_ASCII))
    return Base64.encodeToString(digest, flags)
}
```

INTERVIEW TRAP

Logging in through a WebView. The app can read the password, the user can't verify the real URL, and Google blocks its own sign-in in embedded WebViews. The implicit flow (token in the redirect URL) is deprecated too — use code + PKCE.

FOLLOW-UP

"Which redirect URI?" — Prefer a verified App Link (`https://`). Any app can register the same custom scheme.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Access tokens, refresh tokens, Authenticator

	Access token	Refresh token
Lifetime	Minutes	Days to months
Sent to	Every API call	Only the token endpoint
If stolen	Short damage window	Serious — rotate and detect reuse

```
class TokenAuthenticator(
    private val session: SessionManager,
) : Authenticator {
    override fun authenticate(
        route: Route?,
        response: Response,
    ): Request? {
        // Already retried once with a fresh token: give up
        if (response.priorResponse != null) return null
        val token = session.refreshBlocking() ?: return null
        return response.request.newBuilder()
            .header("Authorization", "Bearer $token")
            .build()
    }
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. How the pieces fit

```
val client = OkHttpClient.Builder()
    .addInterceptor(AuthHeaderInterceptor(session)) // Day 11
    .authenticator(TokenAuthenticator(session))
    .build()
```

- The **interceptor** adds the access token to every request
- OkHttp calls the **Authenticator** when a response is `401`: return a new request to retry, or `null` to give up
- `priorResponse` chains show how many times you already tried — never retry forever

INTERVIEW TRAP

"Store tokens in EncryptedSharedPreferences." — Last year's advice. Jetpack's security-crypto library is deprecated; encrypt with a Keystore-backed key instead (details on Day 19). And keep access tokens in memory when you can.

INTERVIEW TRAP

Refreshing with the **same** `OkHttpClient`: a `401` from the refresh endpoint calls the Authenticator again — recursion. Use a separate client without it.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 12

Senior

Refresh under concurrency · API keys & secrets · Pagination & rate limits



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Five calls get 401 at once

```
override fun authenticate(
    route: Route?,
    response: Response,
): Request? {
    if (response.priorResponse != null) return null
    val sent = response.request.header("Authorization")
    synchronized(lock) {
        val latest = session.accessToken()
        val token = when {
            // Another thread already refreshed: reuse its token
            latest != null && "Bearer $latest" != sent -> latest
            else -> session.refreshBlocking() ?: return null
        }
        return response.request.newBuilder()
            .header("Authorization", "Bearer $token")
            .build()
    }
}
```

- One refresh at a time; everyone who waited reuses the new token
- `synchronized` is fine here: `authenticate()` runs on OkHttp's background threads, never on main



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Why it matters

INTERVIEW TRAP

Five parallel refreshes with **rotating** refresh tokens. The first one wins; the other four present an already-used refresh token. Many servers treat that as theft and revoke the whole session — the user is logged out for no visible reason.

- Refresh failed for good (`invalid_grant`)? Clear the session and expose a "signed out" state the UI observes — don't navigate from an OkHttp thread
- Refresh proactively shortly before `exp` to skip the extra round-trip; keep the Authenticator as the safety net

FOLLOW-UP

"And with Ktor?" — The `Auth` plugin with `bearer { refreshTokens { … } }` handles the 401 → refresh → retry cycle. The same rules apply: a separate client or a marked request for the refresh call, and a clear "signed out" path.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. API keys and secrets: nothing in the APK is secret

- `BuildConfig`, `strings.xml`, native `.so` files, obfuscated strings — all extractable in minutes with standard tools
- The Secrets Gradle Plugin or `local.properties` keeps keys out of **git**, not out of the APK
- Real secrets live on your server: a backend proxy (BFF) calls the third-party API, and the app authenticates to **your** backend with a user token
- Keys that must ship (a maps SDK): restrict them in the provider's console to your package name and signing-certificate fingerprint, and set quotas
- Raise the cost of abuse with Play Integrity or Firebase App Check — verdicts verified **on the server** (Day 20)

RULE OF THUMB

If leaking it would cost money or data, it doesn't belong in the app. Ship identifiers, not secrets.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. The traps around secrets

INTERVIEW TRAP

"Keep API keys in build variants." — Last year's advice. `buildConfigField` values are compiled into the APK in plain text. Build variants separate environments; they don't hide anything.

INTERVIEW TRAP

"Play Integrity API, formerly Play Protect." — Also from last year. It replaced the **SafetyNet Attestation API**. And a verdict only checked on the device can be patched out.

FOLLOW-UP

"Then how does the app call a paid third-party API?" — Through your backend: the app sends its user token, the BFF adds the secret key, applies per-user quotas and caches responses. A leaked user token is revocable; a leaked master key is not.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Pagination: offset vs cursor

	Offset (`?page=3&size=20`)	Cursor (`?after=abc&limit=20`)
Data changes while scrolling	Duplicates or skipped items	Stable
Jump to page N	Yes	No
Deep pages on the server	Slow (skip N rows)	Fast (index seek)

```
class FeedPagingSource(
    private val api: FeedApi,
) : PagingSource<String, PostDto>() {
    override suspend fun load(
        params: LoadParams<String>,
    ): LoadResult<String, PostDto> = try {
        val page = api.feed(params.key, params.loadSize)
        LoadResult.Page(page.items, null, page.nextCursor)
    } catch (e: IOException) {
        LoadResult.Error(e)
    } catch (e: HttpException) {
        LoadResult.Error(e)
    }

    override fun getRefreshKey(
        state: PagingState<String, PostDto>,
    ): String? = null
}
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. Rate limits: be a good client

- `429 Too Many Requests`, usually with `Retry-After`; `503` may carry it too
- Client side: debounce search input (Day 5), deduplicate identical in-flight calls, batch small requests, cache
- The real enforcement is on the server: per user, per key, per IP

```
fun retryAfterMillis(response: Response): Long? {
    val value = response.header("Retry-After") ?: return null
    value.toLongOrNull()?.let { return it * 1_000 }
    val date = response.headers.getDate("Retry-After")
        ?: return null
    val millis = date.time - System.currentTimeMillis()
    return millis.coerceAtLeast(0)
}
```

INTERVIEW TRAP

"Offset pagination is fine for a live feed." — New posts shift every page: duplicates in the list, and a `LazyColumn` with stable keys crashes on the duplicate key. Use cursors (Paging 3 basics on Day 6, `RemoteMediator` on Day 14).

INTERVIEW TRAP

Retrying a `429` immediately — or every client after the same fixed delay. It deepens the overload. Honor `Retry-After`, add jitter.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 12

Architect

Beyond REST · Real-time connections · Security architecture



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Beyond REST: choosing the protocol

	Shape	Android client	Good for	Watch out
REST	Resources over HTTP	Retrofit, Ktor	CRUD APIs, HTTP caching	Over- and under-fetching
GraphQL	One endpoint; client picks fields	Apollo Kotlin (KMP)	Many screens needing different slices of one graph	Client-side caching; errors inside `200`
gRPC	Protobuf over HTTP/2, generated stubs	grpc-kotlin, Wire	Internal, high-throughput or streaming APIs	Needs HTTP/2 end to end; hard to inspect
WebSocket	Full-duplex, long-lived	OkHttp, Ktor	Chat, live collaboration	Reconnects, server scaling, background limits
SSE	Server → client events over HTTP	okhttp-sse, Ktor	Live scores, progress, feeds	One direction only

PICK PER INTERACTION, NOT PER COMPANY

REST for resources, a stream for live updates while the screen is open, push (FCM) for the background. Most real apps use all three.



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. The traps in each

INTERVIEW TRAP

"GraphQL errors arrive as HTTP errors." — Often you get `200 OK` with an `errors` array, possibly next to partial `data`. Check both, or failures render as empty screens.

INTERVIEW TRAP

"gRPC is faster, so use it for the public mobile API." — Binary Protobuf and HTTP/2 help, but you lose easy HTTP caching and human-readable debugging, and every proxy on the path must support HTTP/2. Choose it for a reason, not for speed alone.

FOLLOW-UP

"Do you need GraphQL to tailor responses per screen?" — No. A backend-for-frontend (BFF) with REST endpoints shaped per screen solves the same problem with plain HTTP caching.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Real-time: a WebSocket as a Flow

```
fun OkHttpClient.messages(url: String) = callbackFlow<String> {
    val listener = object : WebSocketListener() {
        override fun onMessage(ws: WebSocket, text: String) {
            trySend(text)
        }
        override fun onFailure(
            ws: WebSocket,
            t: Throwable,
            response: Response?,
        ) {
            close(t)
        }
    }
    val request = Request.Builder().url(url).build()
    val socket = newWebSocket(request, listener)
    awaitClose { socket.close(1000, null) }
}
```

- `awaitClose` closes the socket when the collector goes away — lifecycle for free
- `pingInterval(…)` on the `OkHttpClient` sends heartbeats and detects connections that mobile networks dropped silently



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Reconnect like a senior

```
client.messages(url)
    .retryWhen { cause, attempt ->
        val transient = cause is IOException
        if (transient) delay(backoffWithJitter(attempt))
        transient
    }
```

- **Resume, don't just reconnect:** send the last event ID or sequence number and fetch the gap over REST
- Collect only while the screen is visible (`WhileSubscribed`, `repeatOnLifecycle`); in the background, rely on FCM (Day 16)

INTERVIEW TRAP

Reconnecting immediately in a loop. After a server deploy, a million clients reconnect in the same second — a thundering herd that takes the server down again. Backoff with jitter.

INTERVIEW TRAP

"Keep the WebSocket open in a background service for instant messages." — It drains the battery, and background limits will stop it anyway. Use a high-priority FCM message to wake the app and sync.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Network security architecture: threat → control

Threat	Control
Eavesdropping on public Wi-Fi	TLS everywhere; cleartext off in Network Security Config
Mis-issued certificate for your domain	CT monitoring; pinning with a backup pin where justified
Stolen access token	Short lifetime, refresh rotation, server-side revocation; sender-constrained tokens (DPoP, mTLS)
Modified or scripted client	Server-verified attestation; authorization checked on every call
Leaked third-party secret	No secrets in the APK — the BFF holds them
Scraping, brute force	Rate limits and quotas per user, key and IP
Tokens and personal data in logs	Redacted headers; no body logging in release

INTERVIEW TRAP

"Our API is safe because only our app knows the endpoints." — Anyone can list them with a proxy in minutes. Every endpoint must authorize every request as if it came from a script.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Where the controls live

```
App ── TLS + user token ──▶ BFF / API gateway
                            │ auth · rate limits · shaping
                            ├──▶ internal services (mTLS)
                            └──▶ third-party APIs (secrets)
```

- The **BFF** shapes responses per screen, holds third-party secrets and enforces limits in one place
- **mTLS**: the client presents a certificate too — standard between services; on devices only when you can provision and rotate client keys
- **OWASP MASVS-NETWORK** is a good review checklist: secure all traffic, pin only endpoints you control

SENIOR ANSWER IN ONE LINE

"The network is hostile and so is the client: TLS for the channel, short-lived tokens for identity, the server for every decision — and pin only what you can operate."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 12 · ARCHITECT

Day 12 — cheat sheet

- TLS gives confidentiality, integrity and server authentication — not client identity.
- Cleartext is blocked by default (targetSdk 28+); user CAs aren't trusted (24+). Keep them in `debug-overrides`.
- Never ship a trust-all `TrustManager` or `HostnameVerifier`.
- Pin public keys with a backup pin and an expiration — or don't pin. Old app versions keep pins forever.
- Pinning stops network attackers, not someone inspecting your app on their own device.
- Native sign-in: authorization code + PKCE in Custom Tabs. No client secret, no WebView, no implicit flow.
- Interceptor adds the token; `Authenticator` handles `401` — one refresh at a time, separate client, one retry.
- Nothing in the APK is secret: real secrets stay behind your backend; restrict and attest what must ship.
- Cursor pagination for live data; honor `429` and `Retry-After` with jitter.
- REST for resources, streams while visible, FCM in the background — the server enforces everything.

Day 13 is next → Data Layer, Part 1. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700